

DNS

17 June 2026 12:27 PM

When you type a website address into your **browser**, something almost magical happens. Within seconds, your screen fills with content from servers that might be thousands of miles away.

Behind this seamless experience is a crucial system: **DNS (Domain Name System)**.

It's often called the "phonebook of the internet," but that description barely scratches the surface.

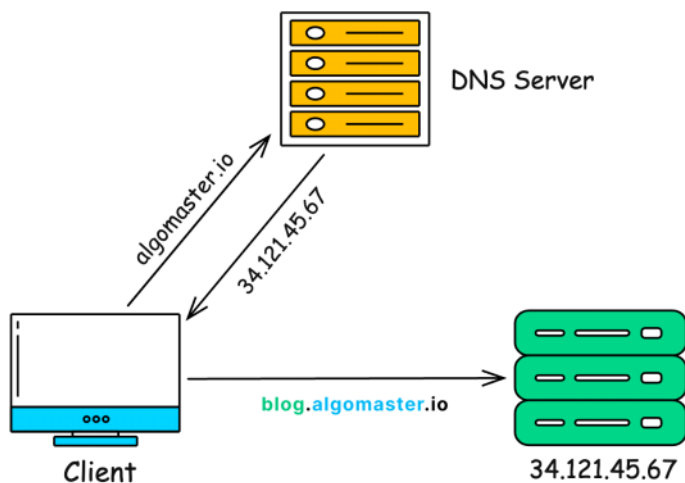
Let's break it down and see how DNS *really* works.

1. Why Do We Need DNS?

On the internet, computers communicate using **IP addresses** such as 104.198.32.55. You can think of them as phone numbers for servers.

Humans, however, are much better at remembering names like google.com. We can't expect users (or even systems) to memorize a string of random numbers for every service they connect to.

This is where the **Domain Name System (DNS)** helps. It acts as a translator between **human-readable domain names** and **machine-friendly IP addresses**.



- You type a URL (algomaster.io)
- DNS maps it to the correct IP (34.121.45.67)
- Your browser connects to that IP
- You see the website

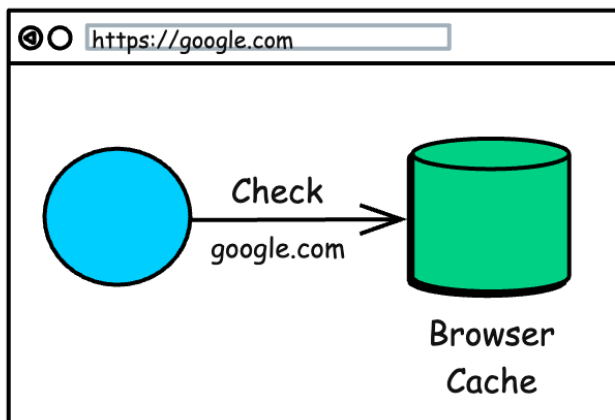
Without DNS, we would all be forced to type raw IP addresses such as 34.121.45.67 into our browsers. That would make using the internet far less convenient.

2. The Journey of a DNS Query

When you enter a domain name (e.g., google.com) in your browser, here's the step-by-step journey of how that name gets resolved into an IP address:

1. The Browser Cache

The browser's first check is its own cache. It checks whether it has recently resolved the domain you're trying to visit.

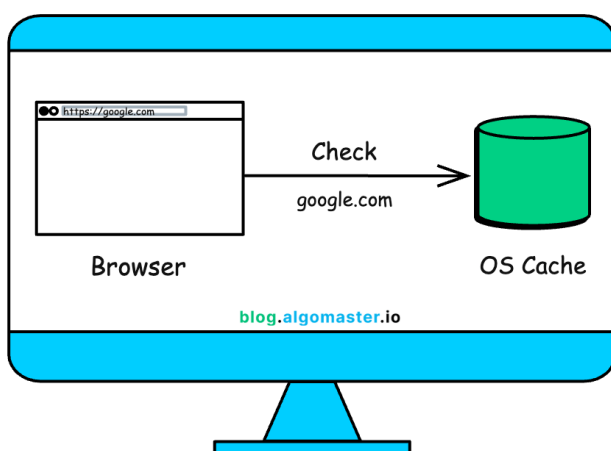


Browser

If so, it uses the cached IP directly. This is the fastest path because no extra work needed.

2. The Operating System Cache

If the browser doesn't know, it turns to your computer's operating system.

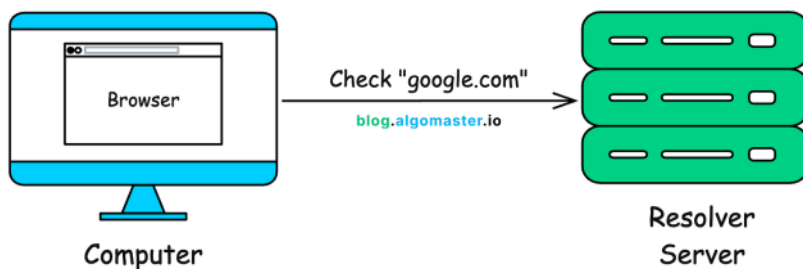


Computer

The OS maintains its own local cache of recent domain lookups, shared across applications. If the record exists here, the operating system returns the IP and the search is complete.

3. The Recursive Resolver

If the OS doesn't have the answer, your computer sends the query to a special server called a **Recursive Resolver**.



This resolver is usually operated by your Internet Service Provider (ISP) or a public DNS service like:

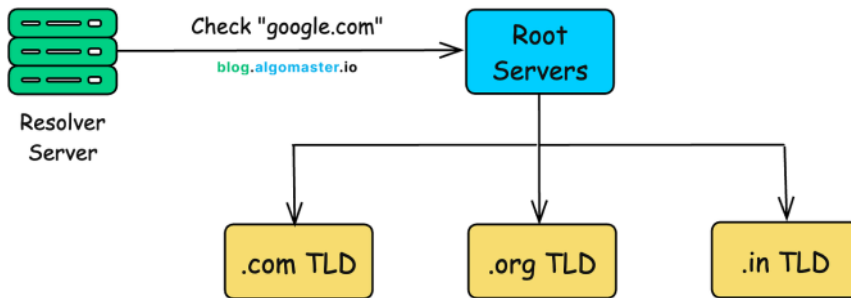
- **Google DNS:** 8.8.8.8
- **Cloudflare DNS:** 1.1.1.1
- **OpenDNS:** 208.67.222.222

The recursive resolver's job is to do all the hard work of hunting down the correct IP address for you. It won't give up until it finds the answer or confirms the domain does not exist.

4. The Root Servers

If the resolver doesn't already have the answer in its cache, it starts its search at the very top of the internet's

hierarchy: the **Root Servers**.



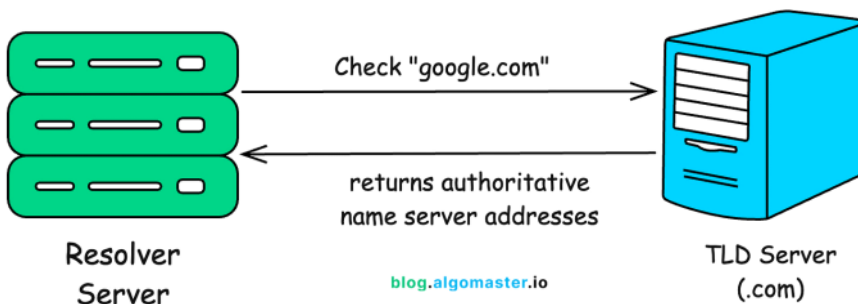
There are only 13 sets of these root servers globally (though they are replicated in hundreds of locations for reliability).

- Root servers don't know the final IP, but they know where to look next.
- They look at the last part of the domain (.com in google.com) and direct the resolver to the appropriate **Top-Level Domain (TLD) server**.

5. The TLD Servers

The resolver now talks to the TLD servers. The **TLD server** manages all domains ending in a specific extension, like .com, .org, .gov, .in, etc.

So, the resolver asks the .com TLD server, "Hey, where can I find information about google.com?"



The TLD server doesn't have the final IP address either. However, it knows which server *is* the official record-keeper for the google.com domain. It points the resolver to that domain's **Authoritative Name Server**.

6. The Authoritative Name Server

Finally, the resolver contacts the **authoritative name server**. This server is the ultimate source of truth for a specific domain. It holds the official DNS records for google.com and knows the exact IP address.



- Authoritative server responds with the correct IP address (e.g., 142.250.183.100).
- These records can include multiple IPs for load balancing and failover.

The authoritative server returns the actual **A record** (for IPv4) or **AAAA record** (for IPv6). It can also return other records depending on the query (e.g., MX for email, CNAME for aliases, TXT for verification).

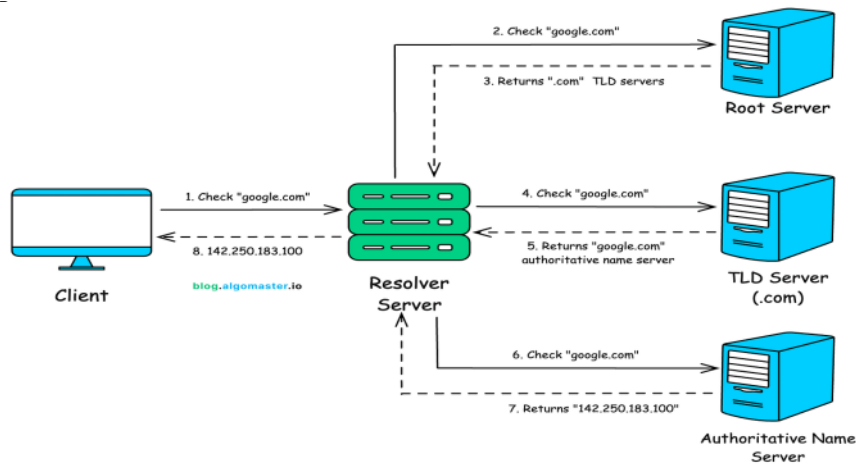
7. Back to the Browser

The recursive resolver now has the IP address. It passes this information back to your computer. Your computer then **caches** this answer so it doesn't have to repeat the whole process next time.

Finally, your browser uses the IP address to connect to Google's server, and the webpage begins to load.

All of this happens in milliseconds.

Summary



3. Types of DNS Records

DNS stores different types of records, each serving a specific purpose:

Record Type	Purpose	Example
A Record	Maps a domain name to an IPv4 address	example.com → 192.168.1.1
AAAA Record	Maps a domain name to an IPv6 address	example.com → 2001:db8::1
CNAME Record	Creates an alias from one domain to another	www.example.com → example.com
MX Record	Specifies mail servers for a domain	gmail.com → mail servers
NS Record	Identifies authoritative name servers	example.com → ns1.example.com
TXT Record	Stores text information (SPF, DKIM, verification)	Email security records
PTR Record	Reverse DNS lookup (IP → Domain)	8.8.8.8 → dns.google
SRV Record	Defines service locations and ports	SIP, LDAP services

4. What Makes DNS Fast and Reliable?

DNS is designed to handle billions of queries efficiently through the following mechanisms:

1. Global Anycast Networks

- Same IP address is advertised from multiple locations worldwide.
- Queries are routed to the nearest available server.
- Reduces latency and improves availability.

Benefits:

- Faster response times
- Better fault tolerance
- Reduced network congestion

2. Redundant Authoritative Servers

- Domains use multiple authoritative name servers.
- Servers are distributed across different regions.
- If one server fails, another can respond.

Benefits:

- High availability
- Fault tolerance
- No single point of failure

3. GeoDNS

- DNS returns different IP addresses based on user location.
- Users are routed to the nearest or most appropriate server.

Benefits:

- Lower latency
- Better user experience
- Compliance with regional regulations

Example:

India User → Mumbai Server
 US User → Virginia Server
 Europe User → Frankfurt Server

4. DNS-Based Load Balancing

- DNS can return multiple IP addresses for a domain.
- Traffic gets distributed among multiple servers.

Benefits:

- Better scalability
- Prevents server overload
- Improves reliability

Example:

example.com
↓
192.168.1.10
192.168.1.20
192.168.1.30

5. Content Delivery Networks (CDNs)

- DNS resolves requests to the nearest CDN edge server.
- Static content is served from locations close to users.

Benefits:

- Reduced latency
- Faster page loads
- Improved user experience

Example:

User → DNS → Nearest CDN Edge Server
→ Images, Videos, CSS, JS